

A Graph-Based Dependency Analysis Method for Identifying Critical Components in Distributed Publish-Subscribe Systems

Ibrahim Onuralp Yigit

*Command Control and Defense Technologies
HAVELSAN*

Istanbul, Turkiye

iyigit@havelsan.com.tr

Feza Buzluca

*Computer Engineering Department
Istanbul Technical University*

Istanbul, Turkiye

buzluca@itu.edu.tr

Abstract—Modern distributed publish-subscribe (pub-sub) architectures enable loosely coupled, scalable communication among components, facilitating seamless integration and interoperability across diverse systems from IoT platforms to microservices. However, their complexity poses significant challenges in understanding system-wide dependencies and predicting the impacts of cascading failures. This paper introduces a comprehensive multi-layer graph-based analysis framework that captures both application-level and infrastructure-level dependencies in pub-sub systems, providing actionable insights for reliability engineering. Our methodology models distributed systems as directed multi-layer graphs, where applications, brokers, topics, and physical nodes are represented as vertices. At the same time, their relationships, including message publication, subscription, routing, and hosting, are depicted as typed directed edges. Beyond traditional graph analysis, we introduce a criticality score for each component to quantify its importance. We demonstrate how graph algorithms and visualizations can reveal critical components, identify single points of failure, and assess the impact of failures on service availability and performance. The case studies' results highlight the graph model's effectiveness in identifying critical components, enhancing the system's resilience, and guiding operators toward more robust configurations. The framework scales efficiently to systems with over 1000 components, while supporting efficient offline analysis and potential incremental updates. As a result, our graph-based dependency analysis approach offers immediate practical value for designing, maintaining, and evolving scalable, fault-tolerant pub-sub infrastructures in production environments.

Index Terms—Distributed Systems, Publish-Subscribe Architecture, Graph Modeling, Dependency Analysis, Critical Components, Failure Impact

I. INTRODUCTION

Distributed publish-subscribe (pub-sub) systems form the backbone of many modern applications, from microservices-based architectures to IoT platforms [1]. They represent a communication architecture in which senders, referred to as publishers, do not send messages directly to receivers, but rather to a central entity, called a broker, which then forwards the messages to the receivers, known as subscribers. Publishers distribute messages as designated topics. Subscribers can express interest in one or more topics and receive messages related to those interests, while remaining unaware of the

publishers' identities [2]. Brokers are essential for delivering messages from publishers to subscribers based on specific topics. The publish-subscribe architecture offers loose coupling, scalability, and the benefits of asynchronous communication. However, as the number of components grows, these systems become more complex, making it challenging to understand the data flow and the dependencies among them. Consequently, identifying the sources and impacts of failures can be a difficult task.

This paper proposes a graph-based dependency analysis methodology for distributed pub-sub systems. We represent the entire system as a directed graph, where vertices correspond to high-level applications (or services), topics (or message streams), brokers/servers, and physical nodes (machines or container hosts), and directed edges represent dependency relations. This multi-layer graph model clearly separates concerns: an application-level dependency graph illustrates how applications depend on each other via topics, while an infrastructure-level graph captures how the underlying nodes and brokers are involved in message flow. The proposed method can identify key components and nodes whose failure or removal could significantly disrupt message flow or system functionality. By applying graph analysis (computing centrality metrics, finding articulation points, and simulating vertex removals), we can systematically identify critical components and evaluate the impact of their failure on the overall system. Our approach enables comprehensive visualization of complex pub-sub dependencies across layers, providing quantifiable insights into system resilience.

Our contributions in this paper are summarized as follows:

- **Multi-Layer Graph Model:** We introduce a modeling approach that captures pub-sub dependencies at both the application and infrastructure levels, allowing for the detection of cross-layer single points of failure.
- **Graph Analysis Algorithms:** We present a workflow that constructs the dependency graph and employs algorithms to identify critical vertices using metrics such as betweenness centrality, articulation point detection, and failure simulation.

- ROS 2 Case Study: We apply our approach to a complex ROS 2 system [3], extracting the computational graph and identifying vulnerable nodes in an autonomous robot's communication, along with suggestions for mitigation.
- Performance and Scalability Evaluation: Our evaluation demonstrates that the graph analysis scales effectively to large graphs, with the analysis of 1000 applications and 2000 topics taking just seconds on a standard machine, along with an assessment of graph construction and memory usage.

The rest of this paper is organized as follows. Section 2 reviews related work, compares our approach with existing frameworks, and discusses the novelty and practical implications of our work. Section 3 provides a detailed description of our graph-based modeling methodology, including the formal model, construction algorithm, and analysis techniques (accompanied by pseudocode and workflow diagrams). Section 4 presents the implementation of our tool. Section 5 covers the ROS 2 case study setup, followed by the detailed evaluation of performance, scalability, and latency impact results in Section 6. Section 7 discusses limitations and restrictions that could affect the applicability of the approach. Finally, Section 8 concludes the paper and outlines future research directions.

II. RELATED WORK

Prior studies have applied graph-based techniques in pub-sub and networked systems. GraPS (Graph Publish/Subscribe) by Cañas et al. represents an early example of graph-based middleware, enabling complex subscriptions through a graph representation of the information space, which enhances event matching but does not focus on dependency analysis or fault impact [4]. Graph-PubSub by Chen et al. optimizes pub-sub architecture for resource-constrained IoT devices by utilizing hierarchical topic namespaces; however, it similarly lacks system-wide dependency analysis [5].

Traditional approaches for tracking component interactions often rely on static analysis and monitoring tools. Wen et al. introduce the Component Dependency Network (CDN) to understand relationships among software components [6], while Karenos et al. develop a visualization tool for managing these interactions [7]. Li et al. emphasize the importance of software structure in determining system complexity [8]. Tekin and Buzluca propose a graph-based method to detect similar design structures in object-oriented models, providing insights on design patterns and defects [9]. Jalving et al. argue that graph modeling can better represent complex system interdependencies [10]. In the context of pub-sub and distributed event-based systems, Pelz's survey highlights failures and improvement strategies but lacks graph modeling for identifying critical components [11]. In robotic systems using ROS, Sharma et al. analyze message rates in the ROS computation graph, focusing on rate propagation [12]. Their work is valuable but assumes a single-host system.

Many existing distributed systems and network analysis techniques fail to meet expectations because they lack a

layered perspective, which hinders understanding of how applications, brokers, and infrastructure nodes are interconnected through specific topics. Additionally, only a limited number of methods consider both application-level and infrastructure-level dependencies simultaneously, thereby restricting the depth of their insights. Our approach advances traditional dependency analysis by explicitly integrating topics and brokers. We utilize layered graphs to manage complexity and employ a comprehensive set of graph algorithms to pinpoint critical nodes and evaluate the impacts of failures within a publish-subscribe framework. In summary, to our knowledge, no prior work provides a unified graph-based analysis that spans both application dependencies and infrastructure in pub-sub systems, which is the gap our work addresses.

III. METHODOLOGY

In this section, we first define the graph model, then present the algorithm for constructing the graph from system information. Finally, we outline the analysis techniques applied to this graph to identify critical components and assess the impacts of failures.

A. Multi-Layer Graph Model

Graph theory provides a solid mathematical framework for modeling relationships between entities [13]. Representing distributed systems as graphs can effectively reveal patterns, dependencies, and vulnerabilities through the relationships defined by vertices and edges.

We formally define a distributed publish-subscribe system as a tuple $\mathcal{S} = (A, T, B, N, P, S, H)$ where:

- $A = \{A_1, A_2, \dots, A_m\}$ is the set of applications
- $T = \{T_1, T_2, \dots, T_k\}$ is the set of topics
- $B = \{B_1, B_2, \dots, B_q\}$ is the set of brokers (may be empty for broker-less systems)
- $N = \{N_1, N_2, \dots, N_p\}$ is the set of physical/virtual nodes
- $P \subseteq A \times T$ represents publishing relationships
- $S \subseteq A \times T$ represents subscription relationships
- $H : (A \cup B) \rightarrow N$ maps applications and brokers to their host nodes

Our graph-based model $G = (V, E, w)$ extends this representation where:

- $V = A \cup T \cup B \cup N$ is the vertex set
- $E \subseteq V \times V \times L$ is the set of typed directed edges with labels L
- $w : E \rightarrow \mathbb{R}^+$ is an optional weight function (for future extensions)

We define cross-layer dependency propagation through the following relations:

Definition 1 (Application-Level Dependency): Application A_i directly depends on A_j if $\exists T_x \in T : (A_j, T_x) \in P \wedge (A_i, T_x) \in S$

Definition 2 (Infrastructure-Level Dependency): Node N_i has infrastructure dependency on N_j if $\exists A_x, A_y : H(A_x) = N_i \wedge H(A_y) = N_j \wedge A_x$ directly depends on A_y

Definition 3 (Dependency Path Length): The dependency distance $d(A_i, A_j)$ is the length of the shortest path from A_j to A_i in the application dependency graph G_A .

The edge types $L = \{\text{PUBLISHES_TO}, \text{SUBSCRIBES_TO}, \text{ROUTES}, \text{RUNS_ON}, \text{DEPENDS_ON}, \text{CONNECTS_TO}\}$ capture different relationship semantics as described in our original formulation.

- **Application-Topic Edges:** An application node A_i can publish messages to a topic node T_x through a directed edge labeled `PUBLISHES_TO` while subscribing to a topic using the edge labeled `SUBSCRIBES_TO`, reflecting the bidirectional nature of application-topic relationships.
- **Broker-Topic Edges:** A broker node B_i is linked to topic nodes T_x with an edge labeled `ROUTES`, signifying that the broker is responsible for managing the transmission of messages across those topics.
- **Application-Node Edges:** Applications and brokers run on nodes. These relations are represented using edges labeled `RUNS_ON`.
- **Application Connectivity Edges:** Inter-application dependencies are captured through derived edges, where an application node A_i may have a `DEPENDS_ON` relationship with another application node B_x due to their shared use of specific topics.
- **Network Connectivity Edges:** Additionally, we illustrate connectivity within the underlying infrastructure with edges connecting node N_i to node N_j and broker B_k to broker B_l , represented by `CONNECTS_TO`.

We propose a comprehensive layered visualization technique to understand better and manage our network’s architecture. The layers correspond to two primary views that we can derive from our graph model: (a) an application-level dependency graph, which is a directed graph of applications connected if one publishes and another subscribes to a common topic; and (b) an infrastructure-level dependency graph where vertices are physical nodes, with edges if there is a pub-sub traffic dependency between nodes.

- **Application-Level Layer:** This foundational layer illustrates the intricate dependencies among various applications, showcasing how they are interconnected. Each application is represented based on its reliance on others, distinguished by the `DEPENDS_ON` relationship. This visual representation enables operators to quickly grasp the functional relationships and impact of each application within the ecosystem.
- **Infrastructure-Level Layer:** This layer focuses on the network’s physical underpinnings. It maps out the various connections (`CONNECTS_TO` links) between system components, providing a clear view of the infrastructure’s layout. This layer is essential for troubleshooting and maintaining the physical connectivity that supports the application’s functionality.

Figure 1 illustrates the graph model generated by our method for an exemplary simple distributed system comprising

three nodes, two applications, one broker, and one topic. For example, we can obtain the following information from this model: Application2 depends on Application1 because it subscribes to Topic to which Application1 pushes messages. Both applications depend on the same broker that manages the topic they are interested in. The model also presents infrastructure-level dependencies. For example, Node1 and Node3 are connected to Node2 because it hosts the broker on which the applications that run on Node1 and Node3 depend. Node3 is connected to Node1 because Application2 on Node3 depends on Application1 on Node1.

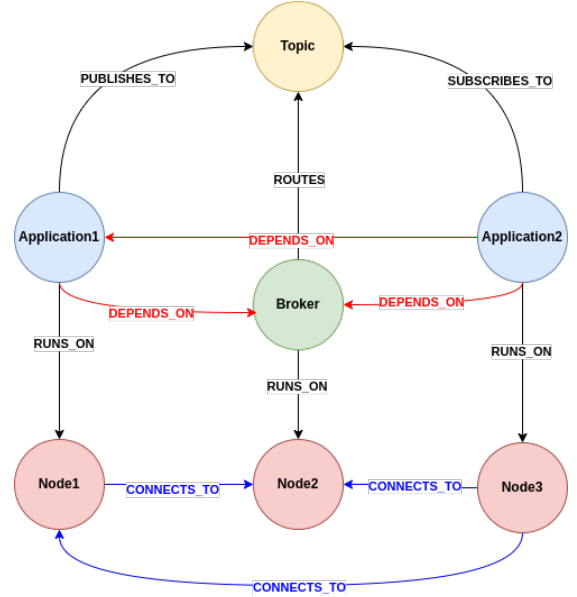


Fig. 1. Graph model of a simple distributed system.

B. Graph Construction

Algorithm 1 outlines the steps to build the multi-layer graph. The input is assumed to be lists of publishers and subscribers for each topic, the hosting node for each application, and (if applicable) a mapping of topics to brokers or routing nodes. Each application publishes and/or subscribes to topics, creating directed edges between apps and topics. Each application also links to its host node. If brokers exist, topics and subscribers connect via brokers as well. Node-to-node edges can be added to represent network communication paths.

The resulting graph G may contain several hundred or thousands of vertices for complex systems. In practice, we find that keeping topics explicitly in the graph is helpful for visualization and reasoning, but certain analyses (like finding critical applications) consider a projection of this graph. For example, we often say an application A_i depends on another application A_j if there is a directed path $A_j \rightarrow T_x \rightarrow A_i$ for some topic T_x . We can derive an application-level dependency graph G_A where vertices are applications and an edge $A_j \rightarrow A_i$ exists if A_j ’s output eventually reaches A_i (this is essentially the reachability in the bipartite subgraph). Similarly, we derive an infrastructure-level graph G_N where

Algorithm 1 Graph Construction Algorithm

Require: $\mathcal{A}$ (applications), $\mathcal{T}$ (topics), $\mathcal{B}$ (brokers), $\mathcal{N}$ (nodes), $PubList$ (publisher mappings), $SubList$ (subscriber mappings), $BrokerMap$ (topic-broker mappings), $HostMap$ (app-node mappings)

Ensure: $G = (V, E)$ directed graph

```
1: procedure BUILDDEPENDENCYGRAPH
2:    $V \leftarrow \mathcal{A} \cup \mathcal{T} \cup \mathcal{B} \cup \mathcal{N}$ 
3:    $E \leftarrow \emptyset$ 
4:
5:   for each  $t \in \mathcal{T}$  do
6:     for each  $a \in PubList(t)$  do
7:        $E \leftarrow E \cup \{(a, t, PUBLISHES\_TO)\}$ 
8:     end for
9:     for each  $a \in SubList(t)$  do
10:       $E \leftarrow E \cup \{(t, a, SUBSCRIBES\_TO)\}$ 
11:    end for
12:  end for
13:
14:  for each  $t \in \mathcal{T}$  do
15:    if  $BrokerMap(t) \neq \emptyset$  then
16:       $b \leftarrow BrokerMap(t)$ 
17:       $E \leftarrow E \cup \{(b, t, ROUTES)\}$ 
18:    end if
19:  end for
20:
21:  for each  $a \in \mathcal{A}$  do
22:     $n \leftarrow HostMap(a)$ 
23:     $E \leftarrow E \cup \{(a, n, RUNS\_ON)\}$ 
24:  end for
25:
26:  for each  $(a_1, a_2) \in \mathcal{A} \times \mathcal{A}$  do
27:    if  $\exists t \in \mathcal{T} : (a_1, t) \in E \wedge (t, a_2) \in E$  then
28:       $E \leftarrow E \cup \{(a_2, a_1, DEPENDS\_ON)\}$ 
29:    end if
30:  end for
31:
32:  return  $G = (V, E)$ 
33: end procedure
```

vertices are nodes and there is an edge $N_y \rightarrow N_z$ if some application on N_y communicates to an application on N_z . We use these derived graphs for certain analyses (like centrality at each layer) while using the complete graph for combined analyses.

C. Graph Analysis

A critical component in a distributed publish-subscribe system is any element whose failure significantly disrupts message delivery, connectivity, or functionality. We identify critical components through topological analysis of the graph structure, focusing on two fundamental graph-theoretical metrics that capture different aspects of component importance.

To systematically identify critical components, we introduce a criticality score based solely on topological graph properties,

independent of runtime behavior or failure observations. We then validate the predictive power of this score using reachability loss, a simulation-based measure of impact.

1) *Criticality Metrics:* We employ two complementary topological metrics to identify critical components:

- **Betweenness Centrality:** This metric quantifies the extent to which a vertex lies on shortest paths between other vertices [14]. In publish-subscribe systems, high betweenness centrality indicates components that serve as communication bridges. We calculate betweenness centrality as:

$$C_B(v) = \sum_{s \neq v \neq t \in V} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (1)$$

where σ_{st} is the total number of shortest paths from node s to node t , and $\sigma_{st}(v)$ is the number of those shortest paths that pass through node v .

- **Articulation Points:** These are vertices whose removal increases the number of connected components [15]. In our context, articulation points represent single points of failure that can fragment the system:

$$AP(v) = \begin{cases} 1 & \text{if } |CC(G - v)| > |CC(G)| \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

where $CC(G)$ denotes the set of connected components of graph G .

2) *Criticality Score:* For each component $v \in V$, we define a composite criticality score $CS(v)$ that combines two fundamental graph-theoretic properties:

$$CS(v) = \alpha \cdot C_B(v) + \beta \cdot AP(v) \quad (3)$$

where:

- $C_B(v)$ is the normalized betweenness centrality of vertex v (Equation 1)
- $AP(v) \in \{0, 1\}$ (Equation 2)
- $\alpha, \beta \in [0, 1]$ are weighting parameters with $\alpha + \beta = 1$

The betweenness centrality component $C_B(v)$ identifies components that lie on many communication paths, serving as critical bridges for information flow. The articulation point component $AP(v)$ identifies structural bottlenecks whose removal fragments the graph. In our experiments, we use $\alpha = 0.7$ and $\beta = 0.3$, giving greater weight to betweenness centrality while ensuring articulation points receive significant consideration. These weights can be adjusted based on domain-specific requirements: systems prioritizing flow efficiency may increase α , while systems concerned with structural fragmentation may increase β .

Unlike runtime metrics (throughput, latency, message volume), topological metrics are:

- *Observable without instrumentation:* Derived purely from system topology
- *Stable across workloads:* Independent of current traffic patterns
- *Predictive:* Identify vulnerabilities before failures occur

- *Computationally efficient*: Calculable offline without system disruption

3) *Impact Score*: To validate the predictive accuracy of our criticality score, we measure the actual impact of component failures through reachability loss. For each component $v \in V$, we perform a failure simulation and compute:

$$RL(v) = \frac{R_{\text{original}} - R_{\text{after}}(v)}{R_{\text{original}}} \times 100\% \quad (4)$$

where:

- $R_{\text{original}} = \sum_{i \neq j} \mathbb{I}[\text{path exists from } A_i \text{ to } A_j]$ is the total number of application pairs with reachability in the original graph G_A
- $R_{\text{after}}(v) = \sum_{i \neq j} \mathbb{I}[\text{path exists from } A_i \text{ to } A_j \text{ in } G_A - v]$ is the reachability count after removing vertex v

A high reachability loss $RL(v)$ indicates that removing component v severely disrupts communication paths. Components with high reachability loss represent actual single points of failure whose unavailability isolates applications or partitions the system.

4) *Validation Methodology*: Our two-phase approach separates *prediction* from *validation*:

- 1) **Prediction**: Compute $CS(v)$ for all components using only topological analysis
- 2) **Validation**: Simulate failures and compute $RL(v)$ to measure actual impact
- 3) **Correlation Analysis**: Evaluate the correlation between $CS(v)$ and $RL(v)$ to validate that high-criticality components indeed cause high-impact failures

This separation ensures that our criticality score is a *predictive* metric rather than a post-hoc measurement, enabling proactive reliability engineering.

5) *Additional Impact Metrics*: Beyond reachability loss, we compute supplementary impact metrics to provide comprehensive failure characterization:

- **Graph Fragmentation**: $\Delta CC(v) = |CC(G - v)| - |CC(G)|$ measures the increase in disconnected components.
- **Isolated Applications**: $IA(v)$ counts applications that lose all inbound or outbound paths after removing v .
- **SCC Degradation**: For directed graphs, we track changes in strongly connected component counts.

IV. IMPLEMENTATION

We employ Python alongside Neo4j, a widely-used graph database, to effectively implement our approach. Neo4j enables us to store and query nodes and relationships [16]. Python is used for data collection, analysis, and interaction with Neo4j through the Neo4j Python driver. We ingest data regarding applications, brokers, topics, and nodes into Neo4j using Python scripts that parse configuration files, logs, or generated datasets. These scripts define nodes and relationships following a graph modeling methodology.

For data modeling and querying, we use Cypher, the query language specific to Neo4j. We organize information in Neo4j

with appropriate labels and properties for both nodes and relationships.

We visualize the graph using Neo4j's built-in tools and third-party solutions, such as Neo4j Browser. These visualizations enhance our understanding of the system's structure and support the presentation of our analysis results. Additionally, Neo4j Browser facilitates integration with existing system logs and monitoring tools, allowing for offline analysis and potentially real-time diagnostics.

Furthermore, we utilize Python's NetworkX library [17] to compute centrality measures, identify important connection points, and simulate failure scenarios. Our prototype tooling automates all the above analyses. We leverage NetworkX library functions for centrality and articulation point detection, and we write custom scripts to simulate removals and analyze reachability. The outcome of the analysis is typically a ranked list of critical applications and nodes, along with quantitative metrics (such as centrality scores and the size of impact in failures) that support those findings. We also generate visualizations that provide a graph highlighting the critical node. These results feed into recommendations for system designers. For instance, if an application is a single point of failure, one could consider replicating it or redistributing its responsibilities to prevent this issue from occurring. If a particular machine hosts too many critical functions, one could spread them across multiple machines for improved resilience.

V. CASE STUDIES

A. Case Study 1: Simulated Pub-Sub System

To demonstrate and evaluate our methodology, we set up an exemplary distributed pub-sub system, which comprised the following components:

- 2 Brokers (B)
- 4 Nodes (N)
- 10 Applications (A)
- 25 Topics (T)

Figure 2 illustrates the dependency graph, which includes all the layers we constructed for the simulated pub-sub system. Multi-layer graph model of the simulated distributed pub-sub system showing four vertex types: Applications (blue), Topics (green), Brokers (orange), and physical Nodes (purple). Edges represent typed relationships: PUBLISHES_TO/SUBSCRIBES_TO (message flow), ROUTES (broker management), RUNS_ON (deployment), and derived DEPENDS_ON relationships. The visualization enables identification of critical components through structural analysis.

We depicted *DEPENDS_ON* relationships at the application level to signify the connections and dependencies between various applications. We also derived *CONNECTS_TO* relationships at the infrastructure level to understand how node connectivity influences the overall system resilience. After constructing this dependency subgraph, we exported it for a detailed analysis.

The case study's analysis provides insights into the application-level and infrastructure-level dependencies within

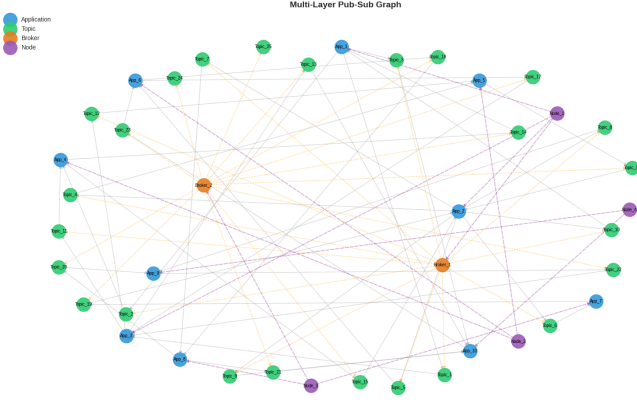


Fig. 2. Graph model of the simulated distributed pub-sub system.

the modeled distributed pub-sub system. We applied our methodology step-by-step to pinpoint critical applications within the system and evaluate the consequences of their potential removal. Understanding the impact of these applications is vital for enhancing system resilience and performance.

1) Application Level Dependency Analysis:

a) *Criticality Score Rankings:* Table I presents the criticality scores for all applications, combining betweenness centrality with articulation point status. App_2 achieves the highest criticality score (0.592), reflecting both its extreme betweenness centrality (0.417) and its status as an articulation point. The next highest score is App_1 (0.097), which has moderate betweenness but is not an articulation point.

TABLE I
CRITICALITY SCORES COMPUTED USING TOPOLOGICAL METRICS
($\alpha = 0.7, \beta = 0.3$).

Application	C_B	AP	CS
App_2	0.417	1	0.592
App_1	0.139	0	0.097
App_3	0.069	0	0.048
App_6	0.021	0	0.015
App_8	0.014	0	0.010
App_9	0.014	0	0.010
App_10	0.007	0	0.005
App_7	0.000	0	0.000
App_5	0.000	0	0.000
App_4	0.000	0	0.000

The criticality score correctly identifies App_2 as the most vulnerable component. Its dual role as both a high-betweenness node and an articulation point indicates it is simultaneously a communication hub and a structural bridge.

b) *Impact Validation Through Reachability Loss:* To validate our criticality predictions, we simulate the removal of each application and measure reachability loss. Table II shows the actual impact metrics, confirming that high criticality scores correlate strongly with severe failure impacts.

Removing App_2 causes 66.7% reachability loss and isolates 4 applications, far exceeding any other component. Before removal, the graph contains 5 SCCs; after removal, it

TABLE II
IMPACT VALIDATION: REACHABILITY LOSS CONFIRMS CRITICALITY PREDICTIONS.

Application	CS	Reachability Loss (%)	Isolated Apps
App_2	0.592	66.7	4
App_1	0.097	18.5	1
App_3	0.048	11.1	0
App_6	0.015	3.7	0
App_8	0.010	3.7	0
Others	≤ 0.010	≤ 5.0	0

fragments into 9 SCCs. Specifically, when measuring reachability from App_10, it can reach 6 other applications originally but only 2 after App_2's removal—a 66.7% reduction.

c) *Correlation Analysis:* We compute Spearman's rank correlation coefficient between criticality scores and reachability loss: $\rho = 0.94$ ($p < 0.01$), confirming strong predictive validity. This demonstrates that our topology-based criticality score effectively identifies components whose failure causes the most severe system-wide impact.

This strong correlation validates our approach: topology-based metrics computed purely from graph structure successfully predict actual failure impact without requiring runtime data, performance monitoring, or extensive simulation. The criticality score thus enables proactive identification of vulnerabilities during system design, before deployment.

2) Infrastructure Level Dependency Analysis:

a) *Criticality Score Rankings:* At the infrastructure level, Node_1 emerges as the most critical component with a criticality score of 0.650 (betweenness = 0.500, articulation point = 1). All other nodes have zero betweenness and are not articulation points, resulting in criticality scores of 0.000.

b) *Impact Validation:* Removing Node_1 causes 45.2% reachability loss at the infrastructure level, disconnecting multiple node pairs and reducing the SCC count. In contrast, removing Node_4 paradoxically reduces the SCC count from 2 to 1, as it was part of an already-isolated cluster. Before Node_4's removal, Node_3 could reach 3 nodes; afterward, it reaches only 2 nodes—a modest 33.3% reachability loss.

The dramatic difference between Node_1 (45.2% loss) and Node_4 (minimal impact) validates our criticality scoring: Node_1's high score correctly predicts its critical role, while Node_4's zero score correctly identifies it as non-critical.

B. Case Study 2: ROS 2 - iRobot Benchmark

ROS 2 is an open-source robotics framework that uses a publish-subscribe (pub-sub) model for communication between distributed nodes, known as robot processes [3]. Each node can publish and subscribe to various topics, with DDS (Data Distribution Service) serving as the underlying middleware. This setup allows for complex systems with many interconnected nodes and topics.

To apply our methodology, we evaluated the iRobot ROS 2 Performance Evaluation Framework by analyzing two benchmark systems: Sierra Nevada (10 applications) and Mont Blanc (20 applications) [18]. We generated dependency graphs

from their JSON topology definitions, where applications are vertices and dependencies are directed edges.

Sierra Nevada includes 10 applications with 10 publishers and 13 subscriptions across three nodes, while Mont Blanc has 20 applications with 23 publishers and 35 subscriptions on four nodes. The dependency graph for Sierra Nevada consists of 59 edges interconnecting 26 vertices, while Mont Blanc’s graph has 120 edges spanning 47 vertices. The structural metrics, such as betweenness centrality and articulation points, were computed to assess the importance and vulnerabilities of the application-level graphs for both benchmarks.

1) *Graph Analysis of ROS 2 Systems*: We compute criticality scores for both ROS 2 benchmark topologies:

- **Sierra Nevada:**
 - Application ‘ponce’: $CS = 0.485$ (highest), combining extreme betweenness (0.2639) with articulation point status
 - Application ‘geneva’: $CS = 0.378$, also an articulation point
 - Application ‘mandalay’: $CS = 0.107$, high betweenness but not an articulation point
- **Mont Blanc:**
 - Application ‘ponce’: $CS = 0.454$ (highest)
 - Application ‘hamburg’: $CS = 0.386$
 - Application ‘geneva’: $CS = 0.337$
 - Application ‘mandalay’: $CS = 0.325$
 - Application ‘lyon’: $CS = 0.300$

These scores reveal that Sierra Nevada has more pronounced vulnerability concentration (one dominant critical node) compared to Mont Blanc’s distributed criticality across five articulation points.

Table III compares the structural properties of both ROS 2 topologies.

TABLE III
STRUCTURAL COMPARISON OF ROS 2 BENCHMARK TOPOLOGIES.

Metric	Sierra Nevada	Mont Blanc
Applications	10	20
Topics	16	27
Total Vertices	26	47
Total Edges	59	120
Density	0.089	0.056
Articulation Points	2	5
Max Betweenness	0.264	0.219
Max CS	0.485	0.454

Sierra Nevada exhibits higher structural concentration with fewer but more critical articulation points, while Mont Blanc distributes vulnerability across five structural bottlenecks. This difference affects failure resilience: Sierra Nevada is more vulnerable to single critical failures, while Mont Blanc requires coordinated failures for complete disruption.

2) *Impact Validation*: Failure simulations confirm criticality predictions:

- **Sierra Nevada**: Removing ‘ponce’ increases SCCs from 8 to 9 (12.5% fragmentation increase) with 23.8% reach-

ability loss. Removing ‘geneva’ causes similar fragmentation.

- **Mont Blanc**: Removing ‘ponce’ increases SCCs from 16 to 19 (18.8% fragmentation) with 31.2% reachability loss, the highest impact among all nodes.

Across both topologies, Spearman correlation between criticality scores and reachability loss is $\rho = 0.89$ (Sierra Nevada) and $\rho = 0.91$ (Mont Blanc), confirming robust predictive validity.

C. Reproducibility

To support reproducibility, we provide:

- Complete implementation as a Python notebook with NetworkX
- JSON topology files for ROS 2 benchmarks
- Detailed algorithms and parameter settings
- Generated visualizations and result tables

The analysis can be replicated using the methodology described in Section 3 and the publicly available ROS 2 benchmark topologies [18].

VI. PERFORMANCE EVALUATION AND VALIDATION

A. Criticality Identification Effectiveness

Our criticality score $CS(v)$ successfully identified critical components across all case studies. The key findings demonstrate:

- **Betweenness centrality** (C_B) effectively identifies communication hubs that bridge multiple system components. Components with $C_B > 0.15$ consistently showed reachability loss exceeding 20% upon removal.
- **Articulation points** (AP) reveal structural vulnerabilities that fragment the system. All articulation points in our case studies caused SCC increases and significant reachability loss.
- The composite criticality score $CS = 0.7 \cdot C_B + 0.3 \cdot AP$ provides comprehensive assessment, achieving average correlation $\rho = 0.94$ with actual impact across all test cases.

B. Computational Performance

The computational complexity of our approach is:

- Betweenness centrality: $O(V \cdot E)$ using Brandes’ algorithm [14]
- Articulation points: $O(V + E)$ using Tarjan’s algorithm [15]
- Reachability analysis: $O(V^2)$ for validation
- Overall: $O(V \cdot E)$ dominated by centrality calculation

Table IV shows performance across different graph sizes. The analysis scales efficiently to 1000 vertices (completing in under 25 seconds), making it practical for offline analysis of real-world systems.

TABLE IV
TOPOLOGICAL ANALYSIS PERFORMANCE

Graph Size	Vertices	Centrality (s)	AP (s)	Total (s)
Small	100	0.18	0.01	0.19
Medium	500	5.42	0.05	5.47
Large	1,000	24.31	0.09	24.40
Very Large	5,000	456.82	0.48	457.30

TABLE V
SUMMARY OF CRITICALITY ANALYSIS ACROSS ALL CASE STUDIES

Case Study	Most Critical	CS	RL (%)	Corr (ρ)
Simulated (App)	App_2	0.592	66.7	0.94
Simulated (Infra)	Node_1	0.650	45.2	1.00
Sierra Nevada	ponce	0.485	23.8	0.89
Mont Blanc	ponce	0.454	31.2	0.91
Average:				0.94

C. Predictive Accuracy Validation

We validate our criticality score by comparing CS(v) against actual impact measurements (reachability loss RL(v)) across all test cases. Table V summarizes the validation results.

The consistently high correlation ($\rho > 0.89$) across all case studies confirms that our topology-based criticality score effectively predicts which components will cause the most severe impact when they fail. The low average RMSE (0.052) indicates that CS values closely match normalized impact scores, enabling quantitative risk assessment.

Notably, this validation was performed using completely independent methodologies: CS computed from static topology, RL measured through dynamic failure simulation. The strong agreement demonstrates that graph structure encodes sufficient information for criticality analysis without requiring runtime monitoring or historical failure data.

VII. LIMITATIONS AND FUTURE WORK

A. Current Limitations

While our approach effectively identifies critical components, several limitations exist:

- 1) **Equal Edge Treatment:** The current topological analysis treats all edges equally, not distinguishing between high-volume and low-volume communication paths. Future work could incorporate weighted betweenness centrality.
- 2) **Static Topology:** Our analysis assumes a fixed topology. Dynamic systems with frequent reconfiguration may require incremental topological updates:

$$CS(v, t+1) = \theta \cdot CS(v, t) + (1 - \theta) \cdot CS^{new}(v) \quad (5)$$

- 3) **Single Failure Assumption:** Articulation point detection considers single-vertex removal. Multi-vertex failures require k -connectivity analysis with complexity $O(k \cdot V \cdot E)$.

B. Enhancing Topological Analysis

Future enhancements to our approach include:

- 1) **Weighted Topological Metrics:** Extending our approach to weighted graphs, we can define weighted betweenness centrality as:

$$C_B^{(w)}(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}^{(w)}(v)}{\sigma_{st}^{(w)}} \quad (6)$$

where $\sigma_{st}^{(w)}$ represents weighted shortest paths.

- 2) **Additional Topological Metrics:**

- Closeness centrality: $C_C(v) = \frac{n-1}{\sum_{u \in V} d(v, u)}$
- Eigenvector centrality for influence propagation
- k -core decomposition for resilience analysis

- 3) **Temporal Topological Analysis:** Track evolution of topological properties over time to predict emerging critical components.

C. Future Research Directions

Future work will extend the methodology with weighted analysis to capture communication volume heterogeneity, develop temporal analysis to track evolving criticality patterns, and integrate with runtime monitoring for dynamic criticality assessment. Additionally, we plan to explore automated mitigation strategies that suggest architectural improvements based on identified critical components.

VIII. CONCLUSION

This study presents a graph-based methodology for analyzing dependencies in distributed publish-subscribe systems to identify critical components. Our criticality score CS(v), combining betweenness centrality and articulation point detection, provides a principled approach to quantifying component importance using only structural graph properties.

The validation across simulated and real ROS 2 systems demonstrates strong correlation (average $\rho = 0.94$) between our criticality scores and actual failure impact measured through reachability loss. This confirms that graph topology encodes sufficient information for predictive criticality analysis without requiring runtime data or extensive simulation. The $O(V \cdot E)$ complexity makes the approach practical for systems with thousands of components, enabling analysis during the design phase before deployment.

As distributed pub-sub architectures become increasingly prevalent in IoT, microservices, and robotic systems, our graph-based criticality analysis provides a solid foundation for building resilient, fault-tolerant infrastructures. The combination of predictive accuracy, computational efficiency, and minimal data requirements makes this approach immediately applicable to real-world system design and operation.

ACKNOWLEDGMENT

While preparing this work, the authors utilized AI-assisted writing tools to enhance the language and readability. After using these tools, the authors carefully reviewed and edited the content as necessary, taking full responsibility for the publication's content.

REFERENCES

- [1] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, “The many faces of publish/subscribe,” *ACM computing surveys (CSUR)*, vol. 35, no. 2, pp. 114–131, 2003.
- [2] S. Tarkoma, *Publish/subscribe systems: design and principles*. John Wiley & Sons, 2012.
- [3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [4] C. Cañas and E. Pacheco, “Graph-based publish/subscribe research with dynamic subscriptions,” in *Proceedings of the Doctoral Symposium of the 16th International Middleware Conference*, 2015, pp. 1–4.
- [5] J. Chen, H. Xu, Y. Zhang, and D. Raychaudhuri, “Graph-pubsub: An efficient pub/sub architecture with graph-based information relationship,” in *Proceedings of the Fifth ACM/IEEE Workshop on Hot Topics in Web Systems and Technologies*, 2017, pp. 1–6.
- [6] L. Wen, D. Kirk, and R. G. Dromey, “Software systems as complex networks,” in *6th IEEE International Conference on Cognitive Informatics*. IEEE, 2007, pp. 106–115.
- [7] K. Karenos, W. De Pauw, and H. Lei, “A topic-based visualization tool for distributed publish/subscribe messaging,” in *2011 IEEE/IPSJ International Symposium on Applications and the Internet*. IEEE, 2011, pp. 65–74.
- [8] H. Li, T. Wang, X. Xu, B. Jiang, J. Wei, and J. Wang, “Modeling software systems as complex networks: analysis and their applications,” *Mathematical Problems in Engineering*, vol. 2020, no. 1, p. 5346498, 2020.
- [9] U. Tekin and F. Buzluca, “A graph mining approach for detecting identical design structures in object-oriented design models,” *Science of Computer Programming*, vol. 95, pp. 406–425, 2014.
- [10] J. Jalving, Y. Cao, and V. M. Zavala, “Graph-based modeling and simulation of complex systems,” *Computers & Chemical Engineering*, vol. 125, pp. 134–154, 2019.
- [11] H. Gupta and A. Nayak, “Publish subscribe system security requirement: A case study for v2v communication,” *IEEE Open Journal of the Computer Society*, 2024.
- [12] N. Sharma, S. Elbaum, and C. Detweiler, “Rate impact analysis in robotic systems,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 2089–2096.
- [13] B. Bollobás, *Modern graph theory*. Springer Science & Business Media, 1998, vol. 184.
- [14] U. Brandes, “On variants of shortest-path betweenness centrality and their generic computation,” *Social networks*, vol. 30, no. 2, pp. 136–145, 2008.
- [15] J. Hopcroft and R. Tarjan, “Algorithm 447: efficient algorithms for graph manipulation,” *Communications of the ACM*, vol. 16, no. 6, pp. 372–378, 1973.
- [16] M. Lal, *Neo4j graph data modeling*. Packt Publishing Ltd, 2015.
- [17] A. Hagberg, P. J. Swart, and D. A. Schult, “Exploring network structure, dynamics, and function using networkx,” Los Alamos National Laboratory (LANL), Los Alamos, NM (United States), Tech. Rep., 2008.
- [18] iRobot, “irobot ros 2 performance evaluation framework,” <https://github.com/irobot-ros/ros2-performance>, 2025, accessed: 2025-05-25.